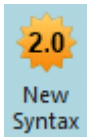
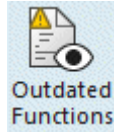


Note

Clicking **New Syntax** in an existing *Method* which you programmed in **SimTalk 1.0 notation** automatically converts the source code to the correct **SimTalk 2.0 notation**.



Click **Find Outdated Functions** on the **Debugger** ribbon tab to find *methods*, *attributes* and *functions* in the source code of all *Methods* in your simulation model.

See also

[Operators and Expressions](#) > [Arithmetic Operators](#)

A Quick Tour Through SimTalk 2.0

A Quick Tour Through SimTalk 2.0

The quick tour through *SimTalk 2.0* gives an overview over the new and improved features.

Remarks

Typically the first example for a programming language prints the words "Hello, World!". In *SimTalk*, you can accomplish this by typing the following into a **Method**:

```
print "Hello World!"
print "Hallo Welt!"
```

This is a complete piece of source code which you can run. You don't need to type in semicolons at the end of each statement as is required in C, in C++, or in JavaScript.

The output in the *Console* looks like this:

```
Console
Hello World!
Hallo Welt!
```

This tour provides you with enough information to start writing code in *SimTalk* by showing you how to approach a variety of programming tasks. Do not worry if you don't understand everything from the

start. Everything that we are going to introduce in this tour is explained in detail in the rest of the *Plant Simulation Help*.

Below we describe:

- **Simple Values**
- **Control Flow**
- **Declare Default Arguments**

Simple Values

You can define simple values in *SimTalk* with *local variables* and *arrays*.

Define a Simple Value with a Local Variable

Use **var** to create a *local variable*:

```
var myVariable := 42
```

A *local variable* must have the same data type as the value you want to assign to it.

If the initial value does not provide enough information, or if there is no initial value, specify the data type by typing it in after the variable, separated by a colon:

```
var myVariable: real := 3.1415
```

Define a Simple Value with an Array

You can create *arrays* using brackets [] and access their elements by entering the index within the brackets.

```
var myIntegerArray: integer[]  
myIntegerArray.append(3)  
print myIntegerArray[1]
```

See also

[Declare Local Variables in the Source Code](#)

[Constant Values](#)

[Colors for Syntax Highlighting](#)

Control Flow

Use **conditionals** and **loops** to define the control flow.

Remarks

Use **if** and **switch** to create **conditionals**.

Use **for**, **while**, and **repeat** to create **loops**.

Parentheses around the *condition* or the *loop variable* are optional, meaning that you can type them, but you do not have to do so.

```
var ages := [34,42,18,44,53,12,63]
var countBelow30: integer
var countAbove29: integer
for var i := 1 to ages.dim
  if ages[i] < 30
    countBelow30 := countBelow30 + 1
  else
    countAbove29 := countAbove29 + 1
  end
next

print "Below 30: ", countBelow30
print "Above 29: ", countAbove29
```

Use **switch** if you have to check for a large amount of different values:

```
var currentDay := 3
switch currentDay
case 1
  print "Monday"
case 2
  print "Tuesday"
case 3
  print "Wednesday"
case 4
  print "Thursday"
case 5
  print "Friday"
case 6
  print "Saturday"
case 7
  print "Sunday"
end
```

Use **while** to repeat a block of code until a condition changes.

```
var n := 2
while n < 10
    n := n * 2
end
```

The condition of a *loop* can also be located at the end instead, ensuring that the loop is executed at least once using the *repeat loop*.

```
var n := 2
repeat
    n := n * 2
until n > 10
```

See also

Colors for Syntax Highlighting

Declare Default Arguments

You can assign **default arguments**, also called **optional arguments**, or **optional parameters** in a *Method* in *SimTalk 2.0*.

Remarks

If the *Method* is called without the argument, the associated parameter will be set to the default value, which is defined in the *Method*.

```
param x: integer := 123
```

The *Method* can be called with or without argument. If the *Method* is called without argument, the parameter named x has the value 123 in the example above.

You can also assign **default arguments** to some of the parameters only. If a parameter has a default argument, all following parameters also have to have a **default argument**.

```
param a: string,  
      b, c: string := "",  
      d: boolean := true
```

The *Method* in the following example can be called with one to four arguments, but not without any argument at all, as no default argument was defined for the parameter named A.

Possible calls are:

```
Method("A")  
Method("A", "B")  
Method("A", "B", "C")  
Method("A", "B", "C", false)
```

Allowed Values for Default Arguments

You can specify the following values for default arguments:

- **Default arguments** only accept **constant values**, i.e., numbers, string constants, the boolean constants true and false, void, and pi.
- **Parameters** of data type *object*, *table*, *list*, *stack*, *queue*, and *any* can only have void as default argument.
- **Parameters** of data type *date* and *dateTime* cannot have a default argument at all.
- **Parameters** of data type *array* can have an empty *array* [] or an empty *JSON object* {} as default argument.